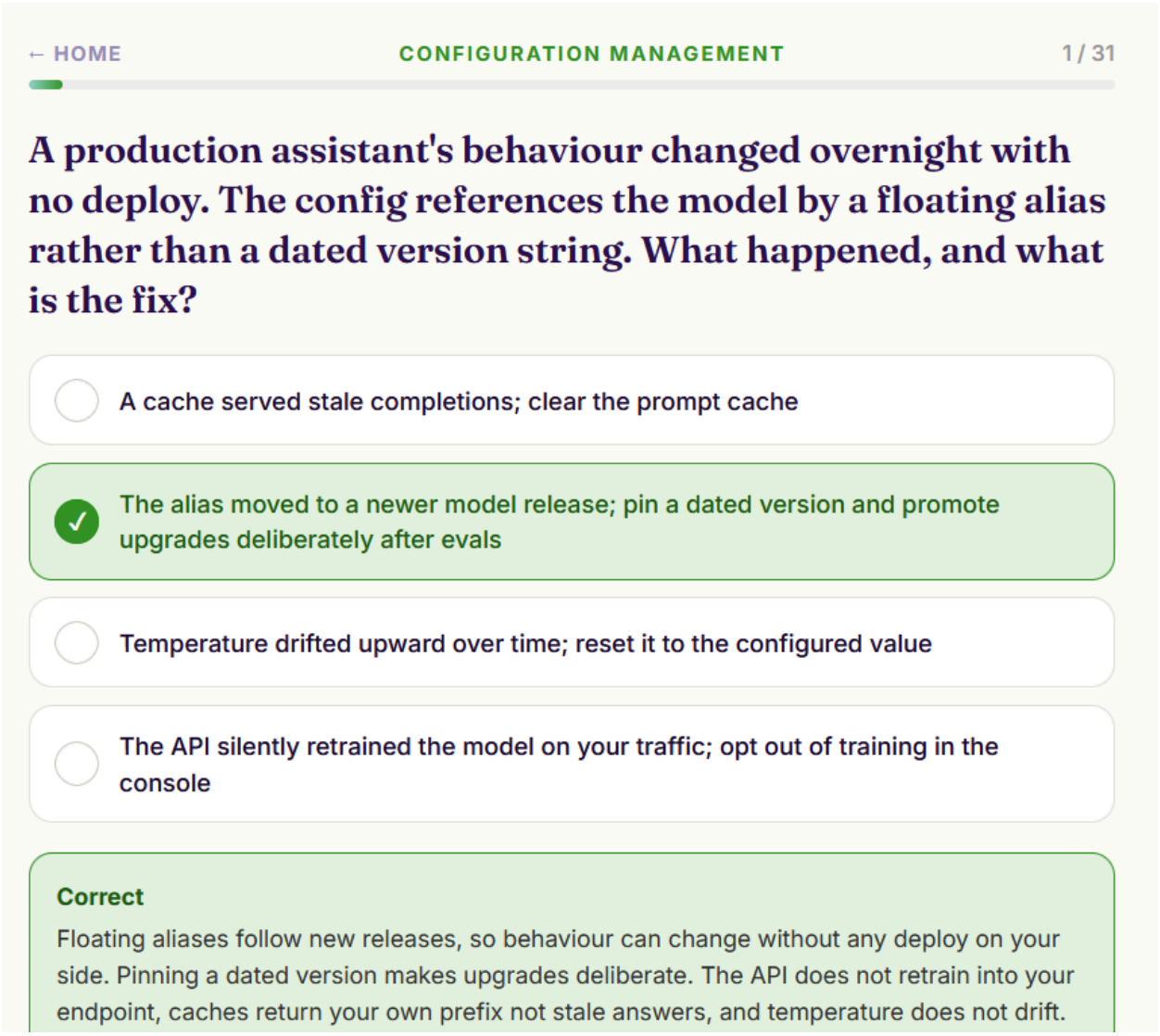
Set 4 : https://claude.ai/public/artifacts/83eee27a-0509-45c1-be08-cc3e8b4cbfc8



Under burst load some calls return 429 rate-limit errors and some return 529 overloaded errors. Correct client behaviour?

- Immediately fail over every call to a different model tier
- Retry 429s in a tight loop since the limit resets quickly, and treat 529s as permanent failures to surface to users at once
- Raise max_tokens so each call does more and you need fewer of them
- ✓ Retry with exponential backoff and jitter, respecting any retry-after guidance, and shed or queue non-urgent work

Correct

Both are transient: back off with jitter and queue what can wait. Tight-looping worsens the burst, 529s are not permanent, blanket tier failover changes behaviour without fixing load, and max_tokens does not reduce request rate meaningfully.

A triage assistant receives full customer records but only needs the message text and product name. Which TWO practices apply? (Pick 2)

Select 2 · chosen 2

- [ ] Encrypt the API key more strongly
- [x] Redact or tokenise identifiers that must pass through, so raw PII isn't in the prompt
- [x] Send only the fields the task needs, stripping the rest before the API call
- [ ] Send the full record but instruct the model not to read the sensitive fields

Correct

Data minimisation means unneeded PII never leaves your system, and pass-through identifiers get redacted or tokenised. Telling the model to ignore fields still transmits them, and key encryption is unrelated to prompt contents.

A day-long agent session must continue, but history has grown near the window limit and old tool outputs are mostly stale. Which lever fits?

- Raise max_tokens
- Increase the temperature to make replies shorter
- Start deleting the system prompt and tool definitions first, since they are the largest single blocks in the window
- ✓ Compact the history: summarise or prune stale turns and tool outputs while preserving decisions and open state

Correct

Compaction summarises or prunes stale history while keeping what still matters. The system prompt and tool definitions are load-bearing, max_tokens governs output not history, and temperature does not manage the window.

A teammate 'slightly improves' the extraction prompt and ships it directly; accuracy quietly drops for a week. What process change prevents this?

- Freeze the prompt permanently
- Restrict prompt edits to senior engineers, whose judgement makes regressions unlikely enough that a formal gate adds little
- ✓ Run the eval suite on every prompt change and gate the deploy on the results, treating prompts like code
- Have the model self-assess whether the new prompt is better

Correct

Prompts are behaviour: version them and gate changes on the eval suite, exactly like code. Seniority does not catch silent regressions, freezing blocks improvement, and self-assessment is not measurement.

A weather agent reports the wrong city. The trace shows the model called get_weather with city='Paris, Texas' while the user asked about Paris, France; the tool returned correct data for what it was asked. Where is the fault?

✗ In the integration layer, since the harness should have corrected the city argument before dispatching the call to the tool

In the tool, which should have guessed the intended city

✓ In the model's argument construction, an output-layer failure, not in the tool or the integration

In the network between harness and tool

Not quite

The trace isolates it: the model built the wrong argument, so the failure is model output. The tool honestly answered what it was asked, harnesses do not silently rewrite arguments, and the network delivered everything correctly.

In a manager/supervisor hierarchy, what is the manager's job?

- Enforce the API rate limits across the team of agents
- Cache the subagents' prompts
- ✓ Decompose the goal, delegate subtasks to specialised subagents, and integrate their results
- Execute every subtask itself while the subagents observe and provide feedback on its work at each step

Correct

The manager decomposes, delegates to specialists, and integrates. It does not do all the work with subagents as reviewers, and rate limiting or caching are infrastructure concerns, not the hierarchy's role.

In an agent harness, a wire-transfer tool must never run on the model's say-so alone. Which tool-usage pattern applies?

- Naming the tool something less inviting
- Prompt-level guidance: a firm system-prompt paragraph explaining the gravity of transfers so the model self-restricts reliably
- Lowering the temperature on calls that mention money
- ✓ An approval pattern: the harness intercepts the call and requires explicit confirmation before dispatching it

Correct

The harness-level approval pattern gates dispatch outside the model's discretion. Prompt guidance is probabilistic, naming is irrelevant, and temperature does not gate execution.

Your RAG pipeline embeds entire 30-page documents as single chunks. Retrieval 'works' but answers are vague and the context fills instantly. What is the fix?

- Switch the embedding model to a larger one
- ✓ Chunk documents into smaller, semantically coherent passages so retrieval returns focused, relevant pieces instead of whole documents
- Keep whole-document chunks but raise the retrieval count so the model has even more full documents to draw from on each question
- Ask the model to ignore irrelevant parts

Correct

Whole-document chunks blunt retrieval precision and blow the budget; smaller coherent passages return exactly the relevant material. More whole documents worsens both problems, a bigger embedder does not fix granularity, and 'ignore the rest' still pays for the rest.

You submitted 80,000 requests to the Message Batches API. Which TWO describe how results work? (Pick 2)

Select 2 · chosen 2

- [ ] If any single request errors, the whole batch is rolled back and refunded
- [x] Individual requests can succeed or fail independently, so results must be checked per item
- [x] You poll the batch for completion and then retrieve results within the processing window
- [ ] Results stream back to you live over a held-open connection as each item completes

Correct

Batches are asynchronous: you poll, then fetch results, and each item carries its own success or error. There is no held-open live stream, and one failed item does not roll back the batch.

Finance asks for per-feature Claude spend, but you currently have no numbers at all. What is the first practical step?

- Enable extended thinking to get more detailed billing
- Estimate from character counts, since tokens are roughly four characters each and the approximation is close enough for accounting
- ✓ Read the usage field returned on each API response and record input/output tokens per feature
- Divide the monthly invoice evenly across features

Correct

Every response reports its token usage; logging that per feature gives real numbers. Character-count estimates drift by content type, an even split hides the actual drivers, and thinking has nothing to do with billing detail.

One API key is shared across dev, staging, and production, held in a team password note. Which TWO changes matter most? (Pick 2)

Select 2 · chosen 2

- [ ] Base64-encode the key inside the note
- [x] Move keys into a secrets manager with access control and rotation, out of shared notes
- [x] Separate keys per environment so a leaked dev key cannot touch production
- [ ] Rename the key so its environment is obvious

Correct

Per-environment keys contain blast radius, and a secrets manager adds access control and rotation that a shared note cannot. Renaming is cosmetic and base64 is encoding, not protection.

You are assembling a streamed response that includes a tool call. Which TWO statements are correct? (Pick 2)

Select 2 · chosen 2

- [x] The end of the HTTP stream is not itself confirmation of a complete message; completion is signalled by the stop event
- [ ] Each delta event is a complete, independently parseable JSON argument object
- [x] A tool call's JSON arguments arrive incrementally across delta events and must be accumulated before parsing
- [ ] Tool calls are never included in streamed responses

Correct

Streaming delivers tool arguments in partial deltas that you accumulate, and completion is signalled by the stop event rather than the socket closing. Deltas are fragments, not standalone JSON, and tool calls absolutely stream.

One incident: a user typed instructions to make the assistant produce disallowed content. Another: a fetched webpage carried hidden text redirecting the agent. How do these differ?

- The first is harmless because the user is authenticated
- They are the same attack, since in both cases text changed the model's behaviour, so one mitigation covers both identically
- ✓ The first is a jailbreak attempt via direct user input; the second is prompt injection via untrusted third-party content, and each is mitigated at a different layer
- The second is impossible if the page loaded over HTTPS

Correct

Jailbreaks come from the user directly; injection rides in through untrusted content the system fetches, and the defenses differ (policy and refusals vs isolation and least privilege). They are not one attack, authentication does not make abuse harmless, and HTTPS says nothing about page content.

A RAG assistant answers confidently but users can't tell which retrieved document supports which claim. Best improvement?

- Lower the temperature so answers sound less confident
- Raise the retrieval count from 5 to 50 passages so more supporting material is always present somewhere in the context
- ✓ Instruct the model to ground each claim in the supplied passages and cite which passage supports it, and validate that cited passages actually exist
- Move the passages into the system prompt

Correct

Grounded, citable answers come from instructing citation against the supplied passages and validating the citations. Fifty passages add noise and cost, temperature does not create

A hard-coded five-step workflow handles invoices, but a new supplier's invoices arrive in unpredictable formats and the workflow breaks on them. What does this signal architecturally?

✓ Inputs now fall outside the codeable path, so the variable part warrants an agent that can decide steps at runtime

The model tier is too small

The workflow should be replaced by a single large prompt

✗ The workflow needs more steps: enumerate every supplier format as its own branch and add a branch each time a new one appears

Not quite

Workflows fit paths you can write down; when inputs fall off the path, the variable portion is agent territory. Enumerating every format is a losing race, tier is not the issue, and a mega-prompt is neither.

Your team is hand-rolling state passing, retries, and branching between eight agent steps, and it's becoming spaghetti. What are frameworks like LangGraph, Strands, or PydanticAI for?

- [x] They provide the orchestration layer: graph or typed control flow, explicit state, and retries, so you stop hand-rolling the plumbing
- [ ] They fine-tune the model so it internalises the eight steps and no orchestration code is needed at all afterwards
- [ ] They are vector databases for retrieval
- [ ] They replace the need for tool schemas

Correct

Agentic abstraction frameworks structure multi-step control flow, state, and recovery. They do not fine-tune models, tools still need schemas, and they are not vector stores.

One endpoint serves both trivial lookups and occasional gnarly multi-constraint planning. You want one configuration that spends reasoning only where warranted. Which option?

- ✓ Adaptive thinking, letting effort scale with the task
- Two separate deployments with a human routing between them
- Fast mode on all calls
- Extended thinking locked to the maximum effort level, so the hard cases are always covered and the easy ones simply finish a little slower

Correct

Adaptive thinking scales effort per task, which is exactly the mixed-load fit. Max-effort everywhere pays thinking cost on trivial lookups, fast mode underserves the hard cases, and human routing does not scale.

Next question →

A support tool reuses one long-running Claude conversation for all customers to 'preserve context.' What is wrong with this design?

- [x] One customer's details leak into another's answers; each user or case needs its own conversation with only its own context
- [ ] It is only a problem for latency, since a long conversation responds slower
- [ ] It breaks prompt caching
- [ ] Nothing, provided the context window is large enough to hold all the customers' histories at once

Correct

Session hygiene: sharing one conversation across customers bleeds one user's data into another's responses, a privacy and correctness failure. A big window does not fix cross-customer leakage, and latency or caching are not the core issue.

You need current exchange rates inside Claude answers. The capability already exists as a maintained internal MCP server other teams use. Build a custom tool anyway?

- Yes, but only as a Skill
- Yes; a hand-written custom tool is always preferable to a shared server because you control the schema end to end and avoid a network dependency
- No; paste today's rates into the system prompt each morning
- ✓ No; connect the maintained server and spend your effort elsewhere, accepting its tool definitions in your context

Correct

A maintained, shared MCP server is exactly the reuse case: connect it rather than duplicating. 'Custom is always better' ignores maintenance cost, pasted rates go stale immediately, and a Skill is instructions, not a live data feed.

Your safety review asks why you have a content policy in the system prompt AND output filtering AND tool-level least privilege. What principle are you applying?

- ✓ Guardrail layering: no single control is relied on, so a bypass of one layer is caught by another
- Compliance theatre required by the auditor
- Defense by obscurity
- Redundancy for its own sake, which the review should flag as waste since the strongest single control makes the other two unnecessary

Correct

Layered guardrails assume any single control can fail and back it with independent ones. Removing layers because one seems strongest recreates a single point of failure, and neither obscurity nor theatre describes overlapping enforceable controls.

Next question →

You wrote a formatting Skill but Claude rarely loads it, even on obviously relevant tasks. First thing to check?

- Whether the skill is written in YAML rather than markdown
- The skill's file size, since larger SKILL.md files rank higher in the loader and small ones are skipped
- The model's temperature
- ✓ The skill's description, since Claude loads a skill by matching the description against the task

Correct

Skills load on demand by description match, so a vague description means it never triggers. Size does not rank skills, SKILL.md is markdown, and temperature is unrelated.

During a Claude Code session, a side-quest (auditing a dependency tree) threatens to flood the main conversation with output. What is the idiomatic move?

- ✓ Hand the audit to a subagent, which works in its own context and returns just the findings to the main thread
- Abort the audit and do it manually later
- Run the audit in the main thread but ask for terse output, keeping everything in one place so nothing is lost between contexts
- Open a second terminal and paste results across

Correct

Subagents isolate noisy side-work in their own context and return a compact result, keeping the main thread clean. Terse-but-inline still spends the main window, and aborting or manual pasting throws away the tooling.

CLAUDE.md files exist at both the user level (~/.claude) and in the project repo. Which TWO are true? (Pick 2)

Select 2 · chosen 2

- [x] Both are loaded, with personal preferences at user level and shared project conventions in the repo
- [x] The project-level file is the one teammates receive via version control
- [ ] The user-level file is committed to the repo automatically
- [ ] Only one file can be active at a time, and the user-level file always wins

Correct

The CLAUDE.md hierarchy layers user-level personal context with version-controlled project conventions, and it is the repo file that teammates share. They are not mutually exclusive, and nothing commits your user file for you.

One tool queries your internal database from the backend; another reads the highlighted text in the user's browser tab. What distinguishes them?

- The difference is only their schema format
- Both are built-in tools

✗ Both must be server-side, since tools always execute on the server that hosts the agent and the browser state is fetched over an API

✓ The database tool is server-side, executed by your backend; the highlight reader must be client-side, executed where the user's state lives

Not quite

Execution location follows where the data lives: backend data is server-side; live browser state can only be read client-side. A server cannot see the user's in-page selection, and this is an execution distinction, not a schema one.

Your Python service uses the Anthropic SDK. A teammate says using the SDK means you're no longer calling the REST API. What is accurate?

- ✓ The SDK wraps the same REST API, adding typed helpers, retries, and auth handling; the underlying HTTP calls are identical
- The SDK is required; raw REST calls are not supported
- The SDK speaks a proprietary binary protocol that bypasses HTTP entirely, which is why it is faster than raw REST calls
- The SDK runs the model locally

Correct

Client SDKs wrap the REST API with ergonomics like typing, retries, and auth; the wire calls are the same HTTP. There is no proprietary bypass, nothing runs locally, and raw REST remains fully supported.

An inspector app sends four photos of the same site and asks Claude to compare them. How is this structured?

- As URLs pasted into the prompt text
- ✓ All four as image blocks in one message alongside the text instruction, so the model sees them together
- Only as a PDF, since multiple raw images are unsupported
- ✗ Four separate conversations, one per image, then a fifth call to merge the four answers, since a message can hold at most one image

Not quite

A single message can carry multiple image blocks with text, letting the model compare them directly. Splitting into per-image calls loses the comparison, pasted URLs are not image input, and raw multiple images are fully supported.

You enabled prompt caching but the hit rate is near zero. The prompt is assembled as: [today's date] + [user profile] + [20k-token policy manual] + [question]. Why?

- ✓ The dynamic date and profile sit before the manual, so the prefix is never identical between calls; move stable content first and dynamic content after it
- ○ Caching needs the Batches API and cannot work on synchronous calls
- ○ Caching only applies to output tokens, so input assembly is irrelevant
- ○ 20k tokens exceeds the maximum cacheable prefix size

Correct

A cache matches on an identical prefix; leading dynamic content breaks the match every call. Put the stable manual first and dynamic values after. Caching works on synchronous calls, 20k is well within cacheable size, and caching is an input-side mechanism.

Free-text user input flows into a prompt that also contains your instructions. Beyond delimiting, what is the input-sanitization step?

- Hash the user text and include only the hash
- Convert the text to uppercase so instructions stand out
- Spell-check and grammar-correct the user text so the model reads it more accurately and is less likely to misinterpret the request
- ✓ Strip or neutralise control-like content in the user text, such as markup that mimics your delimiters or role labels, before it enters the prompt

Correct

Sanitization neutralises content that could impersonate your prompt structure, like fake delimiters or role labels. Spell-checking is cosmetic, uppercase changes nothing about injection, and hashing destroys the content the model needs.

A CI pipeline must run Claude Code on every pull request with no human attached. Which invocation fits?

- ✓ Headless print mode with the prompt passed non-interactively
- Streaming mode in an attached terminal session
- The desktop app on a virtual display
- Interactive mode with an expect-script that answers the confirmation prompts the way a human operator would

Correct

Headless print mode is the non-interactive invocation designed for CI. Scripting fake keystrokes into interactive mode is fragile, and a desktop app or attached terminal is not how pipelines run.

An MCP server should offer users a named, parameterised 'weekly-report' template they can invoke on demand. Which MCP primitive is this?

- ✓ A prompt
- A transport
- A resource
- A tool, since anything the user triggers by name is by definition an action the server performs on their behalf

Correct

Named, parameterised, user-invoked templates are MCP prompts. Tools are model-invoked actions, resources are readable data, and a transport is the communication channel.